

Curso de:

Desarrollo en React JS

Módulo 2:

React intermedio

Unidad 2:

API REST



Presentación

En el desarrollo web moderno, las aplicaciones no funcionan de manera aislada. En efecto, la mayoría de los sistemas requieren acceder, enviar o sincronizar información con fuentes externas de datos, como servidores, servicios de terceros o bases de datos. Este intercambio de datos entre una aplicación *frontend* (cliente) y un sistema *backend* (servidor) se realiza principalmente a través de **APIs REST** (*Representational State Transfer*), una arquitectura ampliamente adoptada para la construcción de servicios web.

Desde el punto de vista de una aplicación React, el consumo de una API REST implica utilizar herramientas del propio lenguaje JavaScript para realizar **solicitudes HTTP**, procesar las respuestas y representarlas en la interfaz de usuario de forma dinámica. Así, esta interacción se convierte en una de las habilidades más importantes en el camino del desarrollo profesional, ya que conecta la lógica de negocio del *backend* con la experiencia de usuario en el *frontend*.

En esta unidad se abordan los fundamentos de la comunicación cliente-servidor, el uso de **fetch** como mecanismo para hacer solicitudes HTTP, y el manejo adecuado de errores mediante bloques **try/catch**. En concreto, el enfoque de este desarrollo está centrado en el rol del desarrollador *frontend*, entendiendo cómo se accede y manipula la información proveniente del servidor para integrarla en una interfaz React.



Objetivos

Que los participantes logren...

- **Comprender** los fundamentos teóricos del modelo cliente-servidor y su aplicación en entornos web.
- **Identificar** qué es una API REST y cómo se organiza su estructura de recursos, rutas y métodos HTTP.
- **Aplicar** el método `fetch` para consumir datos desde una API en una aplicación React.
- **Integrar** datos remotos en componentes mediante *hooks* como `useEffect` y `useState`.
- **Implementar** estructuras de control con `try/catch` para manejar errores durante las solicitudes asincrónicas.
- **Analizar** casos de uso comunes como la obtención de datos, visualización de listas, detalles individuales o interacción con formularios.
- **Adoptar** buenas prácticas para el consumo responsable de APIs en entornos de producción.



Bloques temáticos

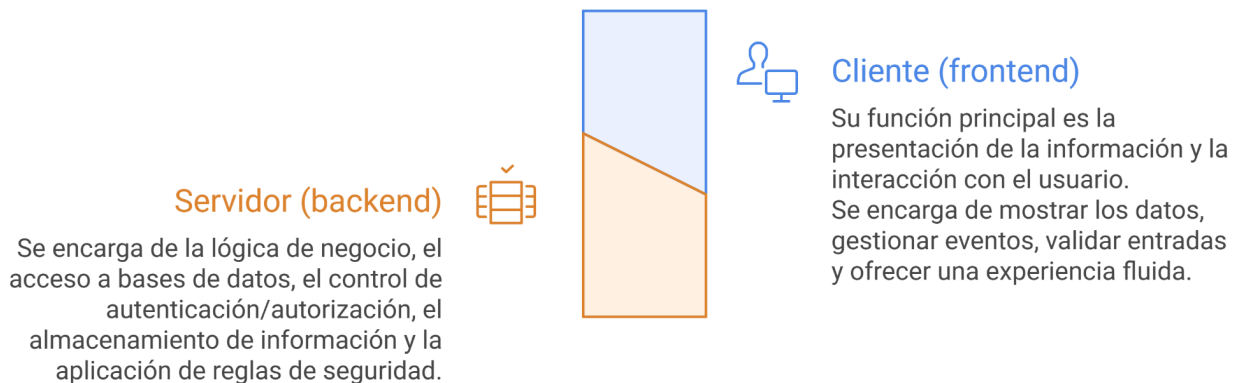
1. Fundamentos de la comunicación cliente-servidor.
 - a. ¿Qué es una API REST?
2. Consumo de API desde nuestra app de React - Fetching.
3. Try/catch.
 - a. Estructura de control para errores.
 - b. Buenas prácticas en el manejo de errores asíncronicos.

Fundamentos de la comunicación cliente-servidor.

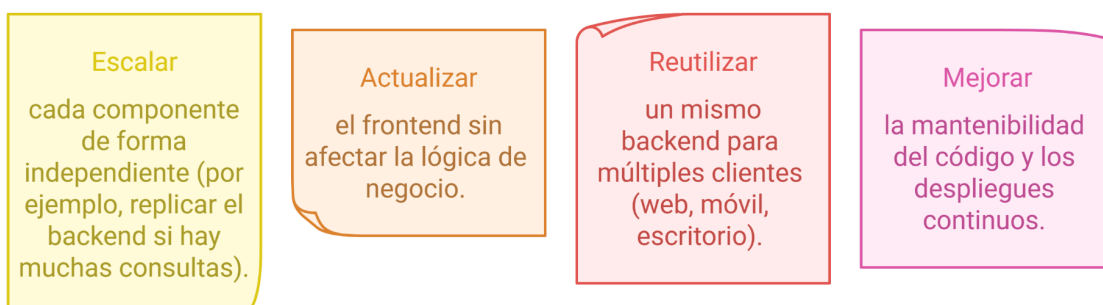
El modelo cliente-servidor es un patrón arquitectónico esencial en el desarrollo de aplicaciones web. En detalle, se basa en una distribución clara de roles: por un lado, el **cliente**, que representa al navegador o la interfaz de usuario y realiza solicitudes a través de la red; por otro lado, el **servidor**, que responde a dichas solicitudes con los recursos o datos solicitados. Esta arquitectura permite separar responsabilidades, escalar proyectos y distribuir tareas entre distintas capas del sistema.

Separación de responsabilidades.

Uno de los principios centrales del modelo cliente-servidor es la separación de responsabilidades, lo que implica dividir las tareas según el rol de cada parte del sistema:



Esta división no solo organiza mejor el sistema, sino que también permite:



Comunicación mediante HTTP.

El protocolo **HTTP** (*Hypertext Transfer Protocol*) es el estándar de comunicación entre cliente y servidor en la web. Este protocolo define una serie de reglas para que los mensajes puedan viajar correctamente desde el navegador hacia el servidor, y viceversa.

Cada **solicitud HTTP** tiene una estructura compuesta por:

- Un **método** (GET, POST, PUT, DELETE, entre otros).
- Una **URL** que identifica el recurso solicitado.
- **Encabezados** (*headers*) que contienen metadatos como tipo de contenido o autenticación.
- Un **cuerpo** (*body*) opcional con datos (generalmente en formato JSON).

A continuación se presentan los **métodos HTTP más utilizados en APIs REST** y sus funciones.

Método	Función
GET	Solicita datos (lectura).
POST	Envía nuevos datos (creación).
PUT	Actualiza datos existentes.
DELETE	Elimina un recurso.

Seguidamente se muestra un **ejemplo conceptual de endpoints REST para un sistema de productos**:



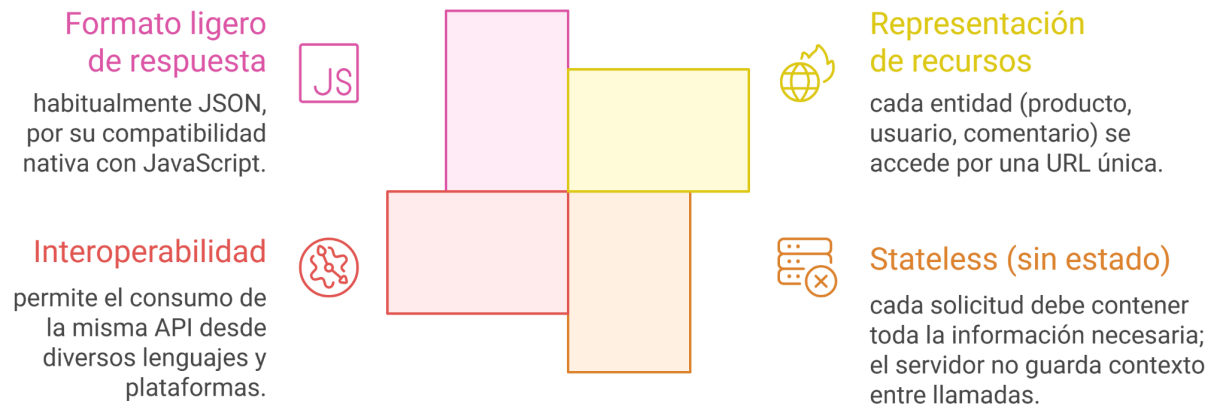
GET	/api/products	→ obtener todos los productos
GET	/api/products/5	→ obtener el producto con ID 5
POST	/api/products	→ crear un nuevo producto
PUT	/api/products/5	→ actualizar el producto 5
DELETE	/api/products/5	→ eliminar el producto 5

Este enfoque basado en recursos es sencillo, uniforme y fácil de consumir desde cualquier cliente.

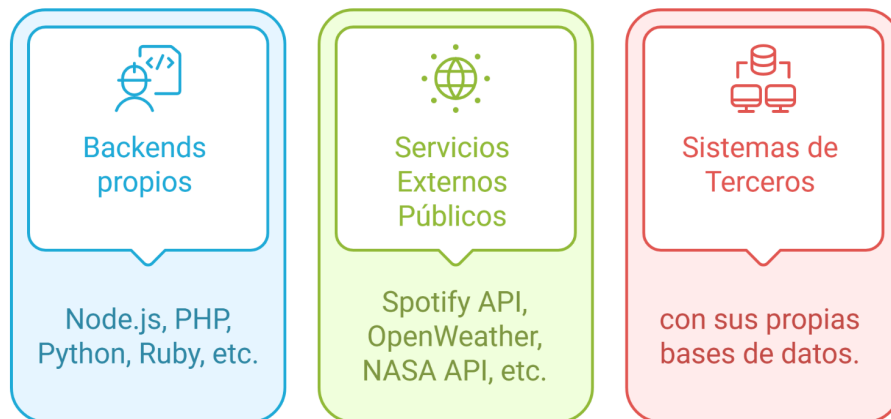
¿Qué es una API REST?

Una **API REST** (*Application Programming Interface* basada en *Representational State Transfer*) es una interfaz de programación que permite a distintos sistemas interactuar mediante el protocolo HTTP. REST no es una tecnología, sino un conjunto de principios de diseño que promueven una comunicación sencilla, sin estado y basada en recursos identificables mediante URLs.

Sus características principales incluyen:



Desde una aplicación React, las API REST actúan como puente hacia:



Consumo de API desde nuestra app de React – Fetching.

Una vez comprendido el modelo de comunicación cliente-servidor y el concepto de API REST, el siguiente paso es **consumir datos desde una aplicación**

React utilizando herramientas modernas como `fetch()` junto con los *hooks* de React.

```
fetch('https://api.example.com/data')  
  .then(response => response.json())  
  .then(data => console.log(data))
```

En React, este patrón se adapta al ciclo de vida de los componentes mediante `useEffect`, que se ejecuta al montar el componente, y `useState`, que maneja el estado de los datos.

```
import { useEffect, useState } from 'react';

function Productos() {
  const [productos, setProductos] = useState([]);

  useEffect(() => {
    fetch('https://fakestoreapi.com/products')
      .then(res => res.json())
      .then(data => setProductos(data));
  }, []);

  return (
    <ul>
      {productos.map(producto => (
        <li key={producto.id}>{producto.title}</li>
      ))}
    </ul>
  );
}
```

Características de una API REST.

Una **API REST** se basa en recursos (como usuarios, productos o pedidos) que se identifican mediante URLs y sobre los cuales se realizan operaciones estándar (GET, POST, PUT, DELETE) para consultar, crear, actualizar o eliminar información, retornando respuestas generalmente en formato JSON o XML.

Cuando se consume una API desde React, el ciclo de vida de la petición puede describirse en los siguientes pasos:

1. **Inicialización del estado:** se declara una variable (ej. **productos**) con **useState** para guardar los datos.
2. **Ejecución de **useEffect**:** se lanza el **fetch** una vez que el componente ha sido montado.
3. **Transformación de la respuesta:** los datos se convierten a JSON.

4. **Actualización del estado:** se actualiza **productos** con la información recibida.
5. **Renderizado de la UI:** React re-renderiza el componente mostrando la información actualizada.

Este flujo garantiza una actualización reactiva y eficiente de la interfaz basada en los datos obtenidos.

Ventajas del enfoque declarativo.

React está basado en un enfoque **declarativo**, lo que significa que el desarrollador describe *qué* debe renderizarse en función del estado, sin preocuparse por *cómo* llegar a ese estado.

Algunas ventajas al integrar **fetch** en este modelo son las siguientes:

- El código es **más predecible** y fácil de mantener.
- Se evita la manipulación directa del DOM.
- Las actualizaciones de datos disparan automáticamente el renderizado correspondiente.
- La lógica de obtención de datos se alinea con el **ciclo de vida del componente**.

Try/Catch.

La comunicación con APIs externas está sujeta a múltiples fuentes de error. Estos errores pueden originarse en problemas de red (como falta de conectividad), fallas en el servidor (caídas, respuestas con códigos de error), errores de cliente (solicitudes mal formadas) o datos inesperados. Por esta razón, **es fundamental incorporar estructuras de control que permitan detectar y gestionar errores de forma eficiente**, garantizando así una experiencia de usuario robusta y predecible.

Bajo este enfoque, el patrón `try/catch` en combinación con `async/await` permite capturar de manera elegante las fallas que puedan producirse durante una operación asincrónica, como una solicitud `fetch()`.

Estructura de control para errores.

El uso de `try/catch` consiste en delimitar un bloque de código que **podría lanzar un error**, capturarlo en el bloque `catch` y decidir cómo actuar frente a ese error (mostrar un mensaje, registrar el problema, evitar un bloqueo de la app, etc.).

En el caso de llamadas a APIs, esto permite verificar:

- Si la solicitud se realizó correctamente.
- Si el servidor devolvió una respuesta válida (`res.ok`).
- Si el formato de los datos recibidos es el esperado.

```
useEffect(() => {  
  const obtenerProductos = async () => {  
    try {  
      const respuesta = await fetch('https://fakestoreapi.com/products');  
      if (!respuesta.ok) throw new Error('Error al obtener datos del servidor');  
      const datos = await respuesta.json();  
      setProductos(datos);  
    } catch (error) {  
      console.error('Ocurrió un error:', error.message);  
    }  
  };  
  
  obtenerProductos();  
}, []);
```

Este patrón es más legible y compacto que el uso excesivo de `then()` y `catch()` encadenados. Además, permite centralizar el manejo de errores y tener una lógica más clara y mantenible.

Beneficios del manejo de errores.

El manejo de errores no solo es una cuestión de buena práctica, sino una **necesidad operativa** en aplicaciones reales. Sus ventajas principales son:

- **Prevención de fallos silenciosos.**

Una solicitud `fetch()` sin manejo de errores puede fallar sin mostrar ningún mensaje ni pista sobre qué ocurrió, dejando al usuario ante una pantalla vacía o un comportamiento inesperado.

- **Mejor experiencia de usuario.**

El manejo explícito de errores permite ofrecer mensajes claros como:

- "Hubo un problema al cargar los datos".
- "La conexión ha fallado. Intente nuevamente".

Esto mejora la percepción de calidad y profesionalismo de la aplicación.

- **Control de flujo y estados.**

Permite estructurar el código para manejar diferentes **estados de la interfaz**:

- **loading**: mientras los datos se están cargando.
- **error**: si ocurre una falla.
- **data**: si los datos llegaron correctamente.

Ejemplo completo con control de estados

El siguiente ejemplo combina el flujo completo de una solicitud a una API con los tres estados más comunes en una aplicación React: **loading**, **error** y **data**.

Este enfoque permite manejar de forma estructurada y predecible las distintas fases que atraviesa una petición asíncrona: desde el momento en que se inicia la solicitud (estado de carga), pasando por la posible aparición de errores (estado de error), hasta la recepción exitosa de la respuesta (estado de datos).

En definitiva, implementar estos estados es fundamental para ofrecer una experiencia de usuario fluida y confiable, ya que permite mostrar indicadores visuales adecuados (como *spinners* o mensajes de error) y actualizar la interfaz en función del resultado obtenido, manteniendo la reactividad y claridad en el código.

```
import { useEffect, useState } from 'react';

function Productos() {
  const [productos, setProductos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchProductos = async () => {
      try {
        const res = await fetch('https://fakestoreapi.com/products');
        if (!res.ok) throw new Error('No se pudo obtener la información');
        const data = await res.json();
        setProductos(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchProductos();
  }, []);

  if (loading) return <p>Cargando productos...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {productos.map(prod => (
        <li key={prod.id}>{prod.title}</li>
      ))}
    </ul>
  );
}
```

Este patrón se convierte en una base sólida para:

- Mostrar **spinners** de carga.
- Renderizar componentes alternativos si hay un error.
- Aplicar renderizado condicional basado en el estado general de la app.

Buenas prácticas en el manejo de errores asincrónicos.

1. Verificar **res.ok** en cada llamada HTTP.

Una práctica común, pero incorrecta, es asumir que el método **fetch** lanzará automáticamente un error ante cualquier respuesta no exitosa del servidor. Sin embargo, esto no es cierto: **fetch** solo lanza una excepción si ocurre un error de red o si la solicitud es abortada por causas técnicas graves, como pérdida de conexión o fallas del navegador.

En cambio, cuando el servidor responde con un código HTTP como 404 (recurso no encontrado) o 500 (error interno del servidor), la promesa de **fetch** se resuelve con normalidad, pero con la propiedad **ok** en **false**. Esto significa que, aunque la solicitud llega a su destino, el servidor responde indicando un problema.

En este sentido, ignorar la verificación de **res.ok** puede dar lugar a errores que pasan desapercibidos, ya que se continúa ejecutando lógica que asume una respuesta válida. Por eso, es fundamental validar explícitamente si la respuesta fue exitosa y, en caso contrario, manejar la situación de forma adecuada, como lanzar un error personalizado o notificar al usuario. Esta verificación es esencial para evitar estados inconsistentes, flujos rotos y para mejorar la trazabilidad de fallas.

2. Utilizar **finally** para estados globales.

El manejo de estados visuales como *spinners* de carga, botones deshabilitados o indicadores de progreso suele formar parte de la lógica de muchas interfaces web. Sin embargo, es fácil caer en el error de habilitar esos elementos al inicio de una operación asincrónica y olvidar

desactivarlos si ocurre un error. Ahí es donde cobra valor el uso del bloque **finally**, el cual permite asegurar que ciertas instrucciones se ejecuten sin importar si la operación fue exitosa o falló.

En contextos asincrónicos, esto es particularmente útil para mantener la integridad de la experiencia de usuario. Por ejemplo, si una operación falla pero el botón de envío sigue deshabilitado o el *loader* queda activo, el usuario podría pensar que la app está congelada. De esta manera, utilizar **finally** como punto único para restablecer esos estados mejora la previsibilidad del código, reduce errores por omisión y evita duplicaciones innecesarias en los bloques de **try** y **catch**.

3. Evitar errores silenciosos.

Uno de los aspectos más críticos para la experiencia del usuario es saber que la aplicación está respondiendo, incluso ante situaciones inesperadas. Cuando ocurre un error y este no se comunica de forma clara, se genera confusión, desconfianza y, en muchos casos, abandono del sistema. Por eso, es importante brindar retroalimentación visible cuando algo no funciona como se esperaba. Esto no significa mostrar mensajes técnicos complejos, sino diseñar mensajes informativos, empáticos y útiles. Por ejemplo, informar que no se pudo cargar el contenido, que ocurrió un problema del lado del servidor, o que se ha perdido la conexión.

Además, un mensaje bien diseñado puede ofrecer alternativas: volver a intentar, contactar con soporte, o seguir navegando con funcionalidades limitadas. El objetivo no es solo alertar sobre el fallo, sino contener al usuario y mantener el control de la experiencia, incluso en escenarios adversos.

4. Desacoplar la lógica de manejo de errores.

Cuando se trabaja en aplicaciones con múltiples puntos de interacción con servidores o APIs externas, es común que surjan patrones repetitivos: validación de la respuesta, tratamiento de errores, actualización de estados, etc. Si cada componente maneja esto de manera aislada, el código se vuelve redundante, difícil de mantener y propenso a errores inconsistentes.

La solución a este problema es el desacoplamiento: mover la lógica común hacia funciones reutilizables, ya sea mediante servicios dedicados o *custom hooks*, según la arquitectura utilizada. Este enfoque permite concentrar en un solo lugar todo el tratamiento de errores, facilitando su evolución, testeo y documentación. Por ejemplo, si se desea implementar una lógica adicional, como mostrar un *toast* o registrar un error en un sistema de monitoreo, solo es necesario hacerlo en un punto centralizado. Además, este desacoplamiento mejora la legibilidad del código en los componentes, que pueden enfocarse en la lógica de presentación sin distraerse con detalles técnicos de bajo nivel.

5. Loggear errores críticos.

No todos los errores necesitan llegar al usuario, pero sí deben llegar al equipo de desarrollo. Especialmente en producción, es esencial contar con mecanismos de monitoreo que permitan detectar fallas en tiempo real, sin depender exclusivamente de reportes manuales. Para ello, es una buena práctica registrar los errores críticos que afectan el funcionamiento general de la aplicación.

Estos errores pueden ser capturados y enviados automáticamente a servicios especializados como Sentry, Datadog o similares, que permiten obtener información contextual sobre el fallo, incluyendo el entorno, el

usuario afectado, el momento en que ocurrió y los eventos previos al error. Este tipo de trazabilidad facilita una respuesta rápida y efectiva por parte del equipo de desarrollo, acortando los tiempos de resolución y reduciendo el impacto negativo en los usuarios. Por el contrario, ignorar esta práctica implica perder visibilidad sobre el comportamiento real de la aplicación en producción y confiar ciegamente en que "todo funciona", lo cual rara vez es el caso en entornos reales.

Extensión: visualización de múltiples estados.

Con pequeñas variaciones, se puede mejorar la visualización condicional de un componente según el estado:

```
if (loading) {  
  return <div className="spinner">Cargando...</div>;  
}  
  
if (error) {  
  return <div className="error-box">Ocurrió un error: {error}</div>;  
}  
  
if (productos.length === 0) {  
  return <div>No hay productos disponibles.</div>;  
}
```

Este enfoque mejora la robustez de la interfaz y permite a los usuarios **entender en todo momento qué está ocurriendo**.



Bibliografía utilizada y sugerida

Fielding, R. y Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content (RFC 7231)*. Internet Engineering Task Force (IETF).
<https://datatracker.ietf.org/doc/html/rfc7231>¹

Google Developers. (2014). *Web Fundamentals and Web Starter Kit: Resources for Modern Web Development*. Google Web Fundamentals.
<https://developers.googleblog.com/en/web-fundamentals-and-web-starter-kit-resources-for-modern-web-development/>²

Gupta, L. (2025). *What is REST?*. REST API Tutorial. <https://restfulapi.net/>³

MDN Web Docs. (s.f.-a). *Generalidades del protocolo HTTP*. Mozilla Corporation. <https://developer.mozilla.org/es/docs/Web/HTTP/Overview>⁴

MDN Web Docs. (s.f.-b). *Introducción a las APIs web*. Mozilla Corporation.
https://developer.mozilla.org/es/docs/Learn/JavaScript/Client-side_web_APIs/introduction⁵

MDN Web Docs. (s.f.-c). *Fetch API*. Mozilla Corporation.
https://developer.mozilla.org/es/docs/Web/API/Fetch_API⁶

¹ Especificación técnica oficial de los métodos HTTP y sus usos.

² Introducción al modelo cliente-servidor en el desarrollo moderno.

³ Guía práctica sobre cómo diseñar y consumir APIs RESTful de forma correcta.

⁴ Descripción clara del protocolo HTTP, sus métodos, encabezados y códigos de estado.

⁵ Explicación detallada sobre qué es una API, cómo funciona y cómo consumirla desde el navegador.

⁶ Documentación completa sobre la función **fetch**, sus métodos, encabezados, manejo de errores, uso con JSON, etc.

MDN Web Docs. (s.f.-d). *Promise*. Mozilla Corporation.

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Promise⁷

⁷ Referencia fundamental sobre el funcionamiento de promesas, **then**, **catch**, y **finally**.